

CLOUD NATIVE
COMPUTING FOUNDATION

Slurm-on-K8s For AI Platform

Adaptive HPC Scheduling on
Cloud Native Infrastructure
For AI Batch Job

Agenda

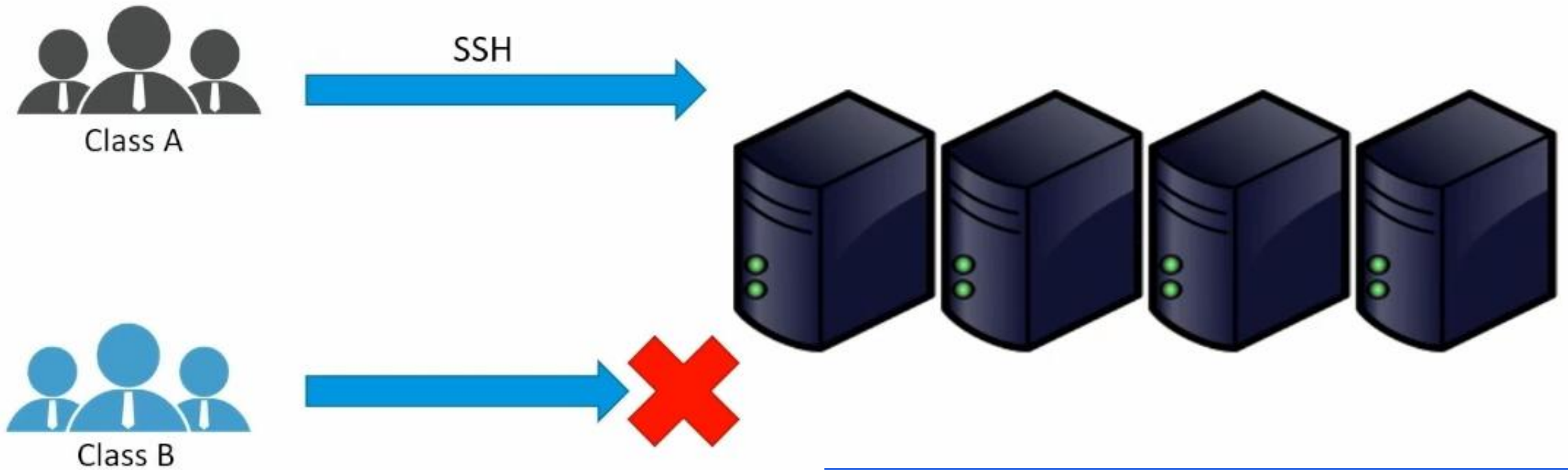
-  **1 Introduction**
-  **2 System Architecture**
-  **3 Demo & Evaluation**
-  **4 References**



01. Introduction

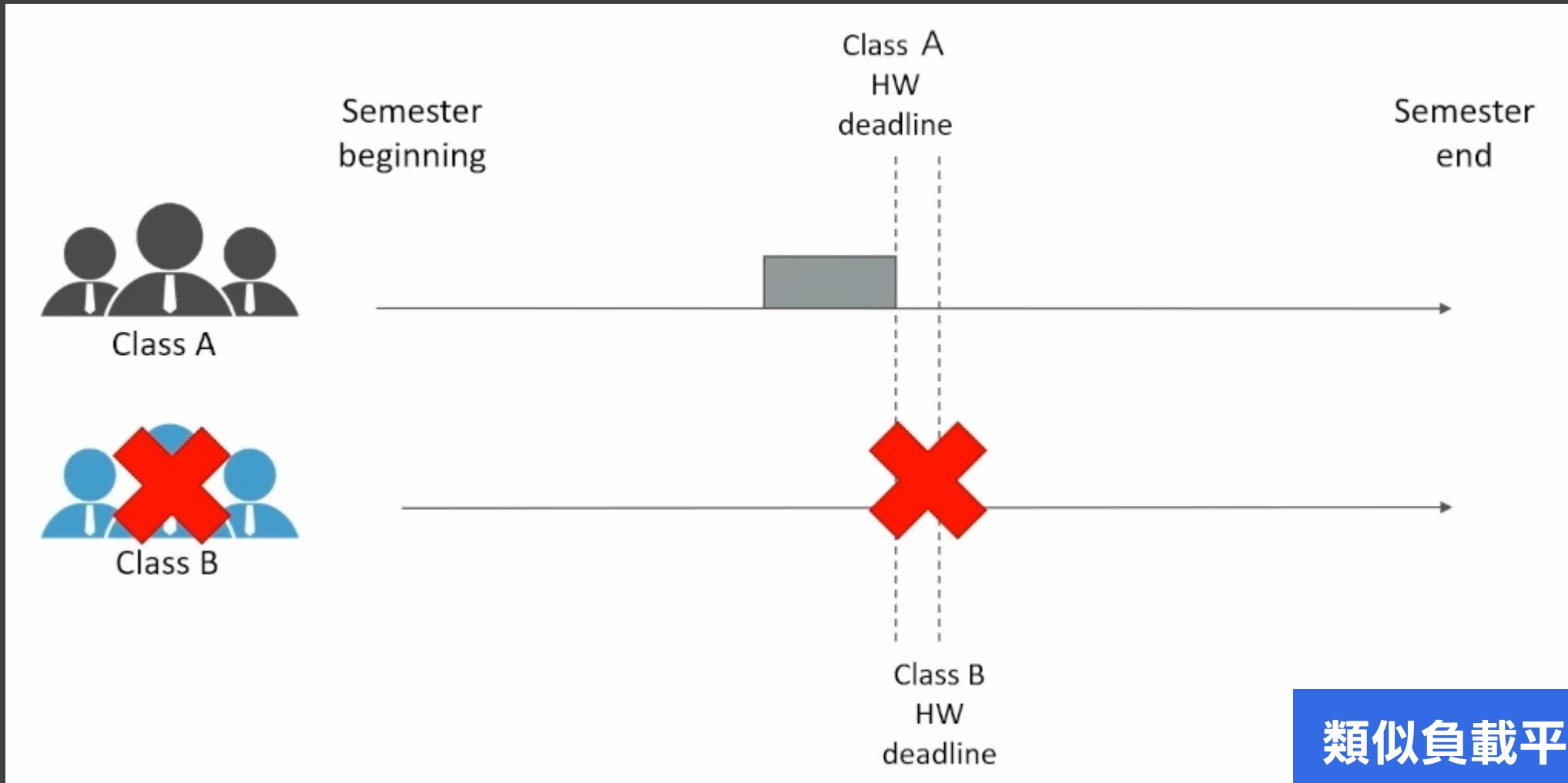
專題簡介

Example Scenarios



有沒有辦法多台機器給多個人使用？

Example Scenarios



類似負載平衡的概念

Slurm

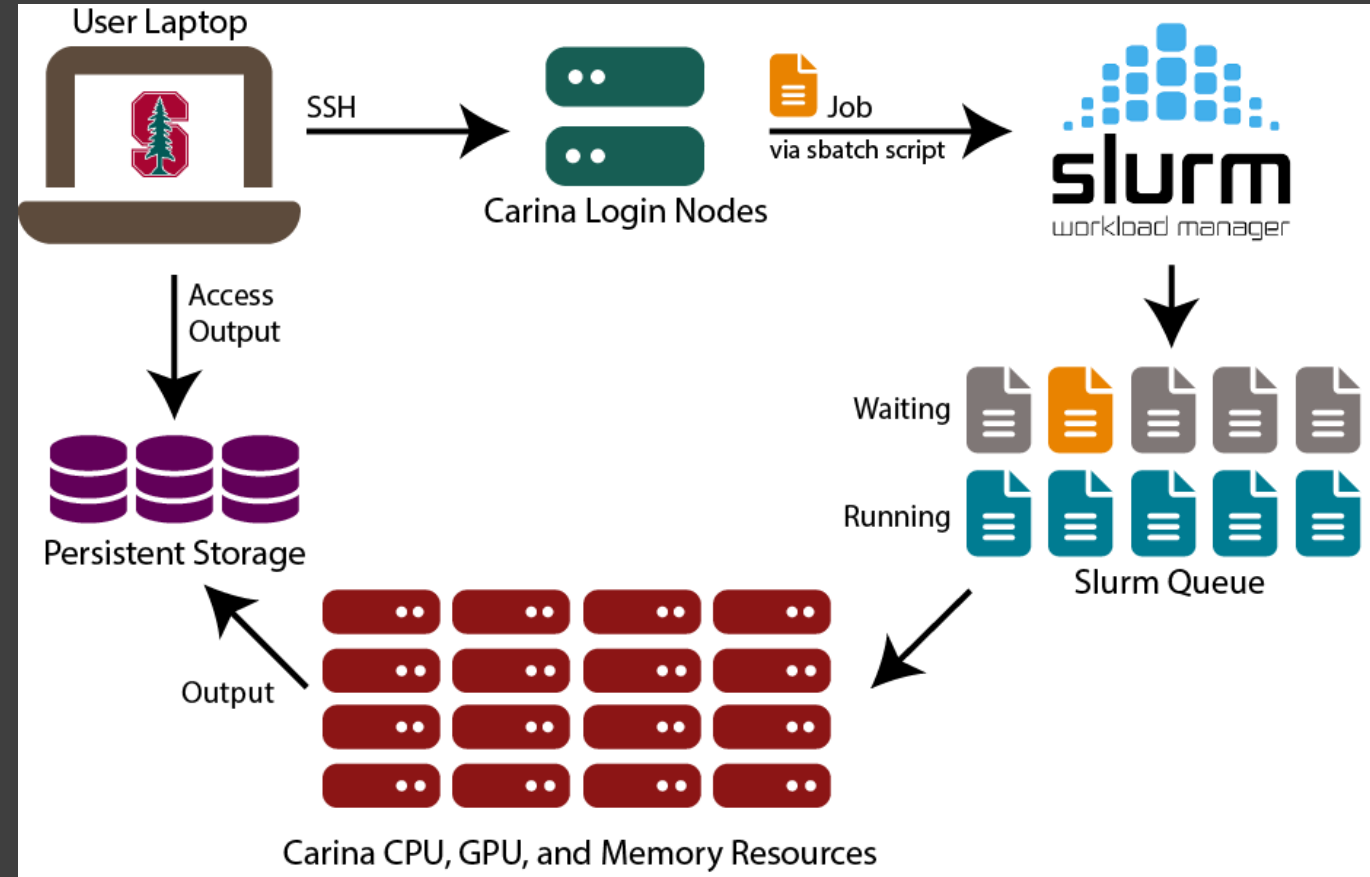
What is Slurm ?

- Slurm = Simple Linux Utility for Resource Management
- An open source system
 - 管理工作：資料分配、排程、執行



Roles in Slurm

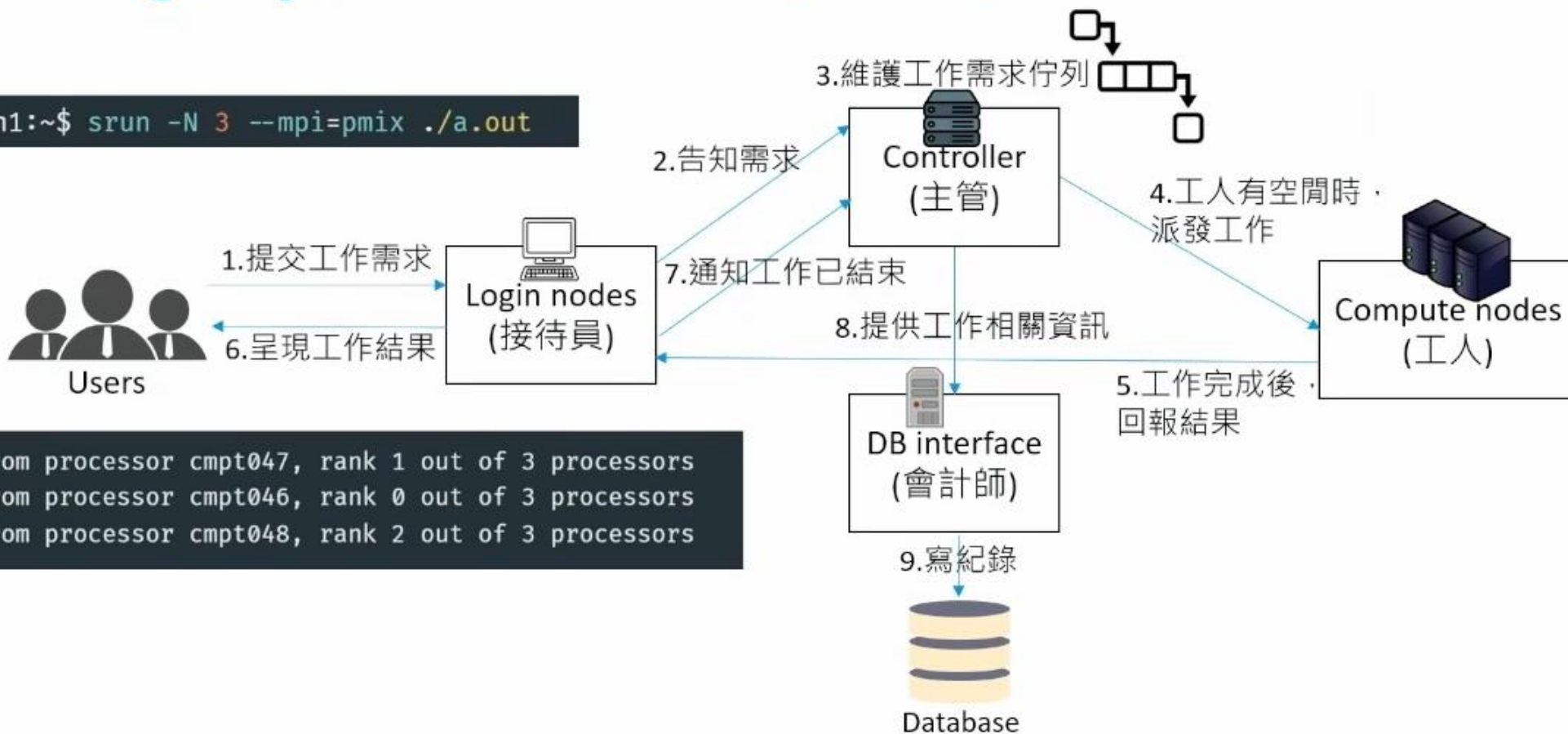
- Login nodes = 接待員/客服
- Controller = 主管
- Compute nodes = 工人
- DB interface = 會計師



Running a job in Slurm (srun)

Running a job in Slurm (srun)

```
bjhuang@login1:~$ srun -N 3 --mpi=pmix ./a.out
```



```
Hello world from processor cmpt047, rank 1 out of 3 processors
Hello world from processor cmpt046, rank 0 out of 3 processors
Hello world from processor cmpt048, rank 2 out of 3 processors
```

Kubernetes

What is Kubernetes ?

- Kubernetes (K8s) = Containers Orchestration Tool
- An open source system
 - 管理容器：啟動、運算資源、排程、自動部署
 - 大公司架設服務很愛用

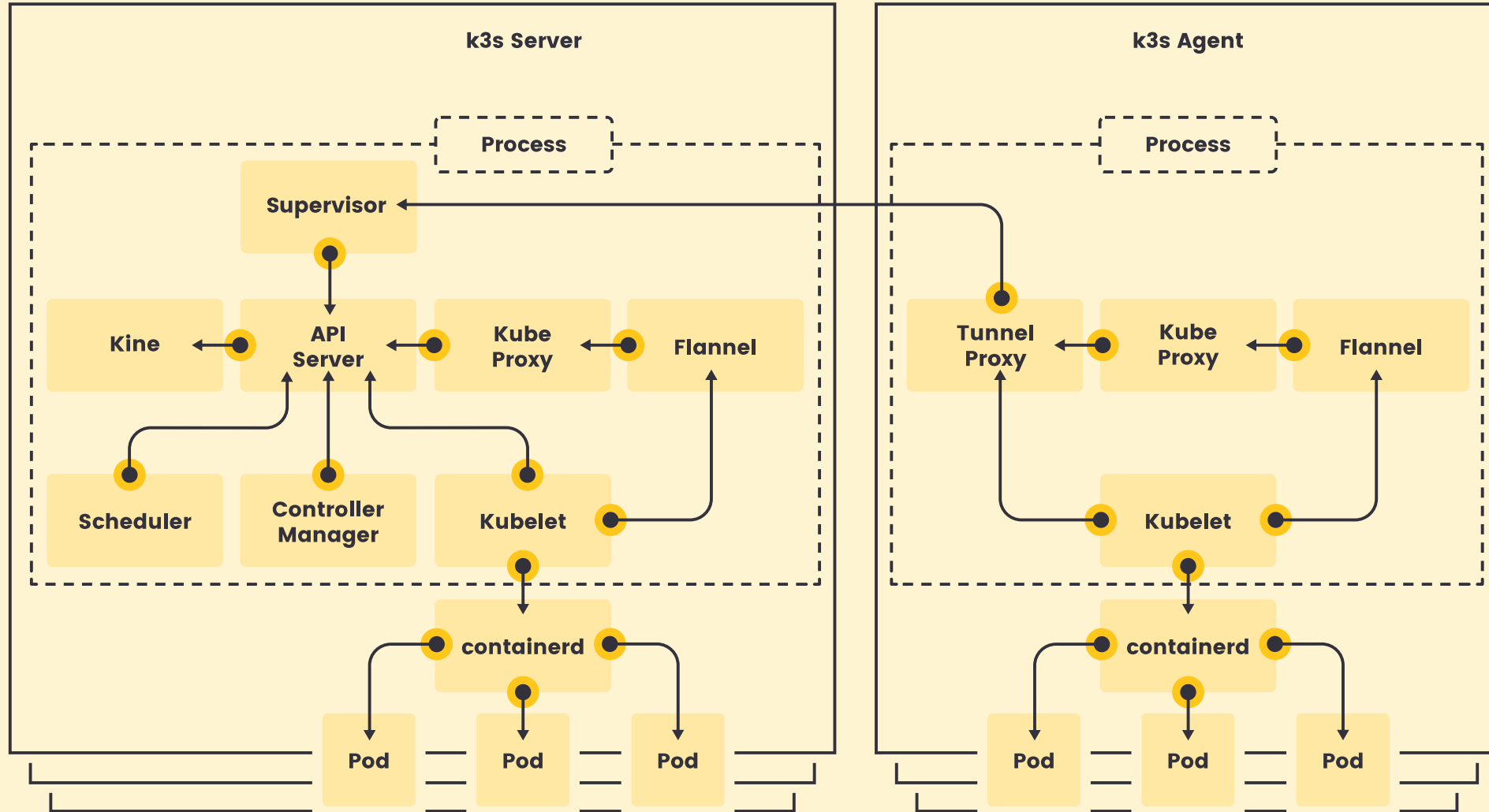


Kubernetes Distribution

- 本來是用 kind，但硬體支援度有極限，改成使用 K3s
- 建立在 Kubernetes 標準安裝之上的一個輕量發行版
- 由 Rancher Labs 開發，通過 CNCF 認證。



K3s



Background

Motivation

- Kubernetes 很擅長資源編排與彈性伸縮，但對 HPC / AI 訓練工作負載的語意支援有限。
 - Slurm 很擅長批次排程、資源配額與叢集治理，但傳統部署通常偏靜態，對雲端環境的節點生命週期不夠友善。
- ⇒ 打造一個讓多位使用者共用的批次 AI 工作平台



Benefits

- 提升資源利用率
 - Slurm 可以透過將工作排程到 K8s 叢集中的可用資源
- 簡化的工作負載管理
 - 一律透過 Slurm 介面，用於管理 ML 工作負載
- 提升可擴展性
 - Slurm 可以透過將機器學習工作負載排程到多個 K8s 叢集來擴展

Slurm on Kubernetes

- Slurm 決定誰用資源、什麼時候用、用多少
 - 計算資源排程問題
- Kubernetes 決定資源怎麼存在、怎麼啟動、怎麼管理
 - 動態提供 Slurm 需要的運算節點
 - 當硬體調整的時候不用重新調整 Slurm 設定
 - 依賴版本管理、網路設定

Our Feature !



- 擴容、擴縮、低功耗、節點壞掉重啟
- 客製化排程演算法、運算資源分配
- 支援多種 Job
- 近年來企業很重視安全，可以裝個龍蝦



Reliability



Performance



Lifecycle Unity



Observability



02. System Architecture

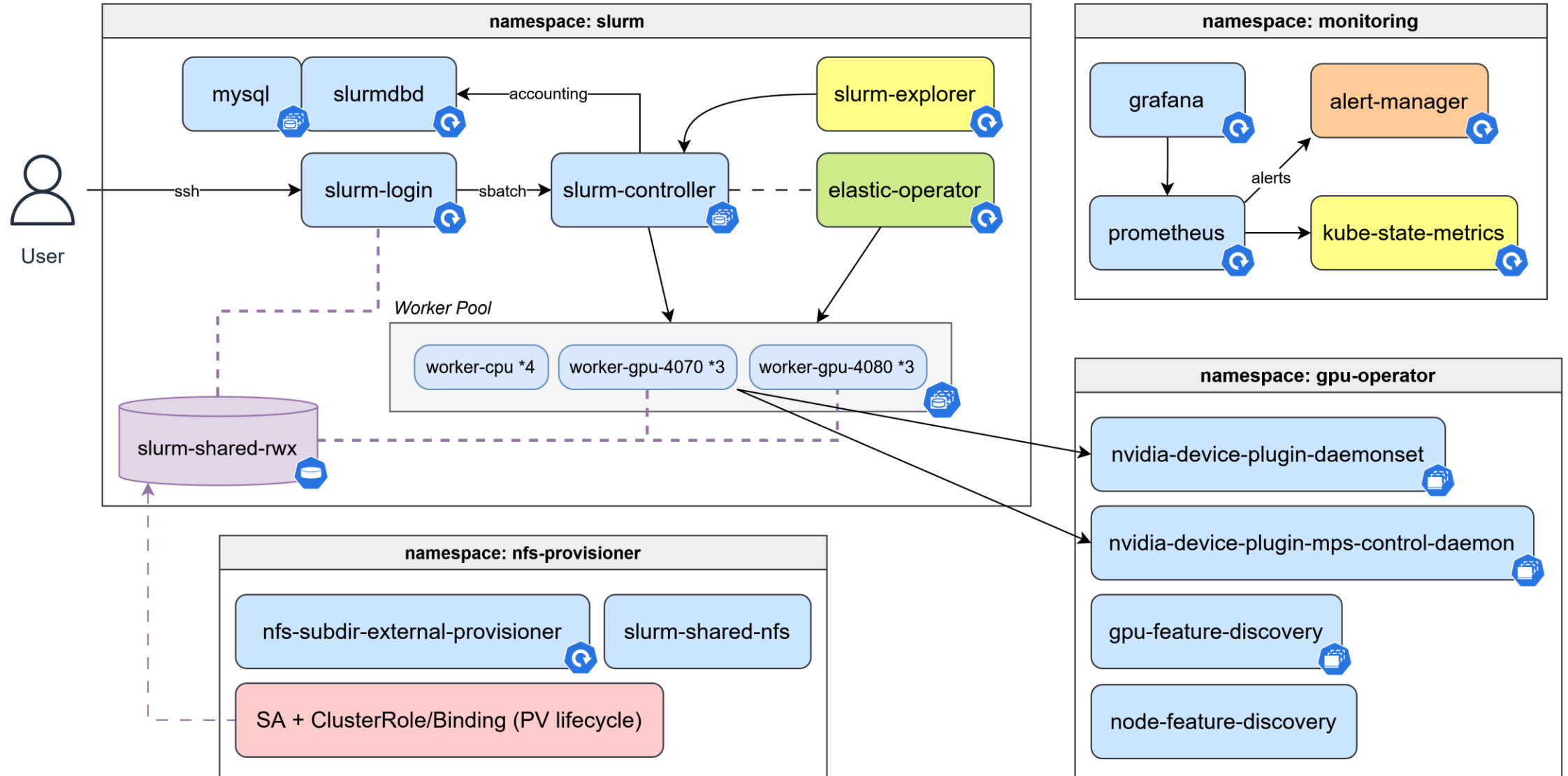
系統設計與架構

Development Environment



- Ubuntu 24.04 + K3s v1.34 + RTX 4070 (lab)
- Operator : Python 3.11-slim + kubectl CLI
- Nvidia GPU Operator

K8s (K3s) Cluster



Slurm GRES

- Slurm 的通用資源管理機制，讓 Slurm 能夠追蹤和排程 CPU/Memory 以外的資源 (e.g. GPU)
- 知道每個 node 上有哪些 GPU (型號、數量)
- 讓使用者提交 job 時指定需要的 GPU 類型和數量
- 確保 GPU 資源不會被過度分配

GPU Resources

Multi-Process Service (MPS)

- 在 GPU 上創建多個 context，將 GPU 的資源分配給多個 process
- 每個進程擁有一個獨立的 context，可以獨立執行其計算任務
- 提高多容器共享 GPU 的 QoS
- 不管 VRAM、記憶體頻寬、L2 Cache、編解碼器、Copy Engines
 - export `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0,3072` (slot_0 3072 MB)

NVIDIA GPU MPS



請求 + constraint	RTX 4070 分配 SM	RTX 4080 分配 SM	適合工作
<code>--gres=mps:50 --constraint=gpu-rtx4070</code>	~24 SM (50%)	—	中型推論 (7B LLM serving)
<code>--gres=mps:25 --constraint=gpu-rtx4070</code>	~12 SM (25%)	—	小型推論 (image classifier)
<code>--gres=gpu:rtx4070:1</code>	48 SM (整卡)	—	訓練 (≤12 GB VRAM)
<code>--gres=gpu:rtx4080:1</code>	—	76 SM (整卡)	訓練 (≤16 GB VRAM)
<code>-N 2 --gres=gpu:1</code>	48 SM	76 SM	2-GPU DDP

Job Migration

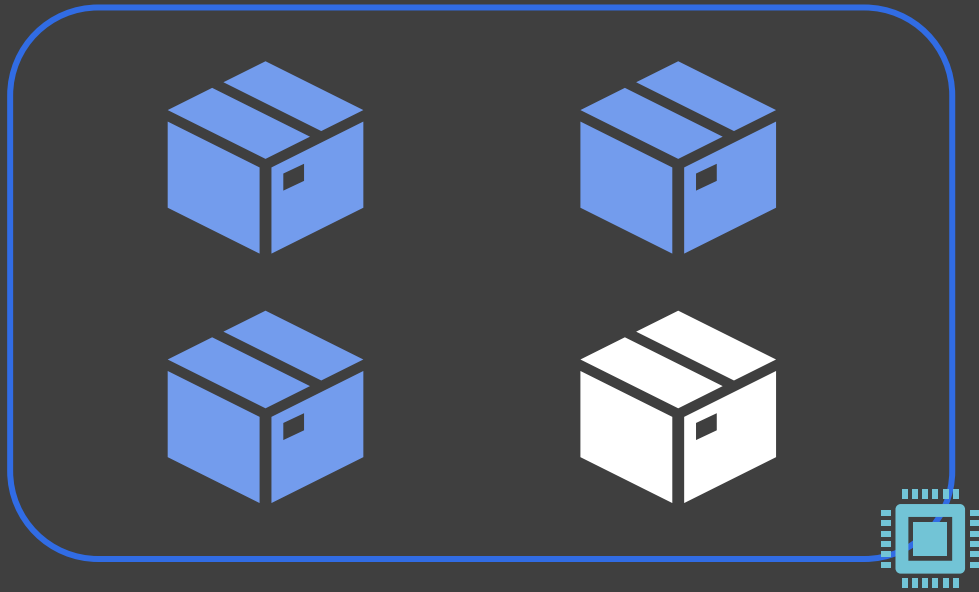
Job A



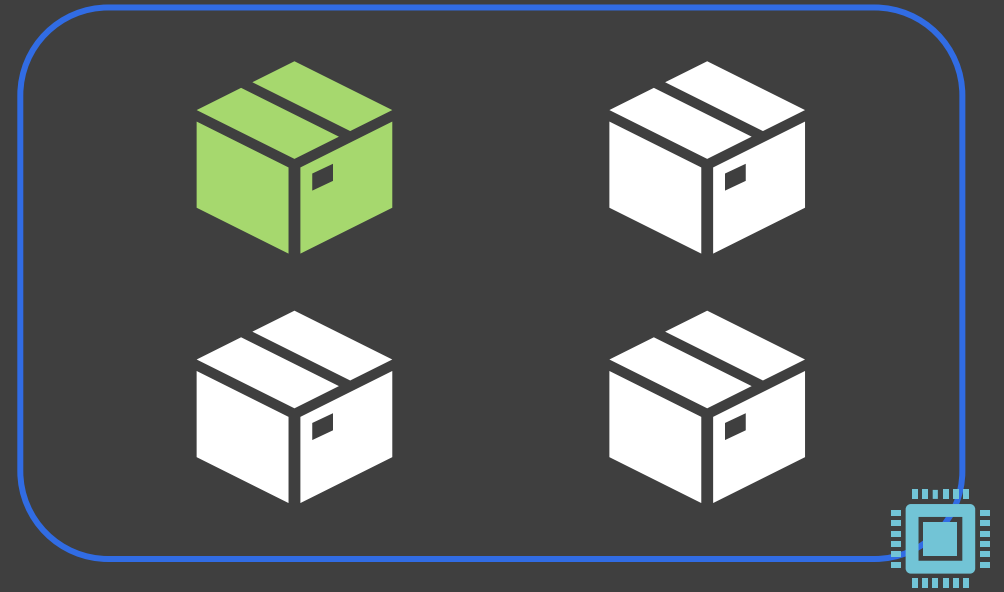
Job B



Job C

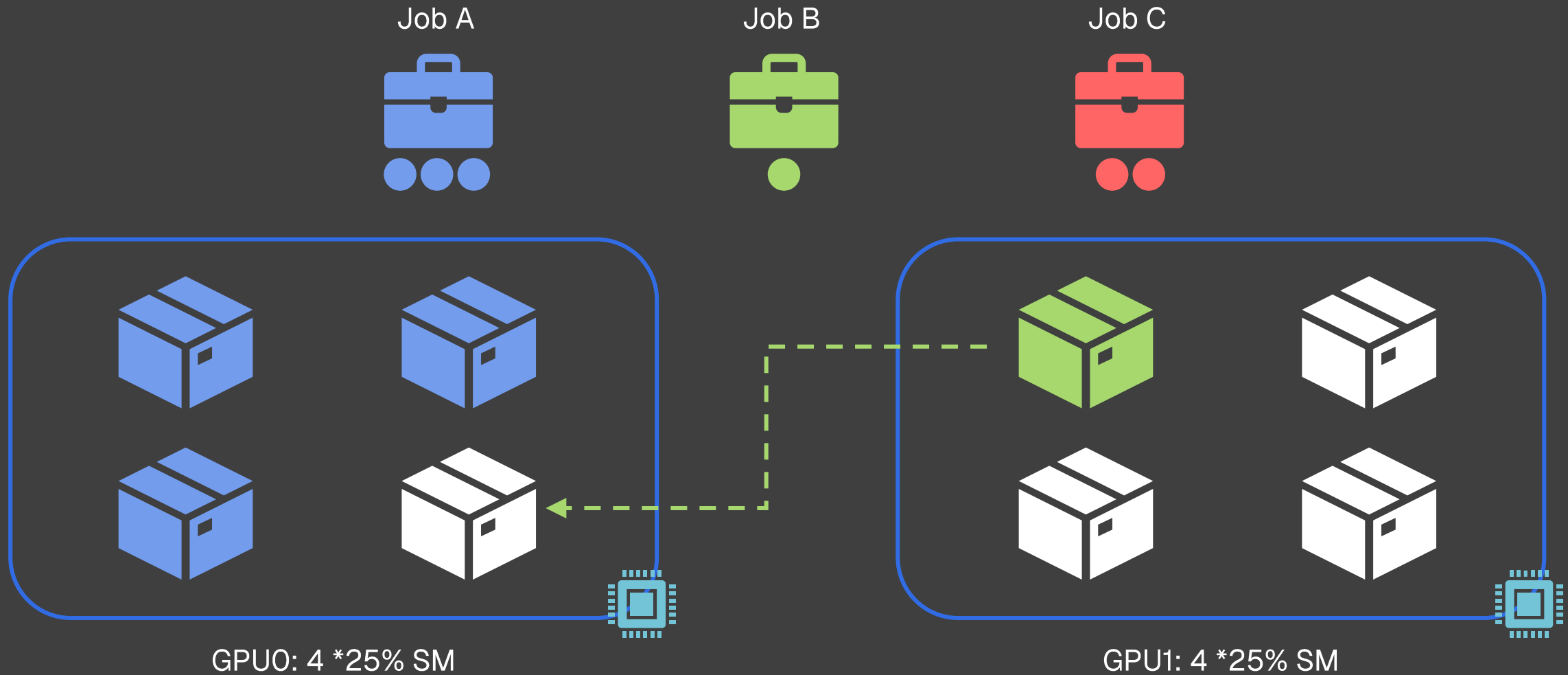


GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Migration



Job Migration

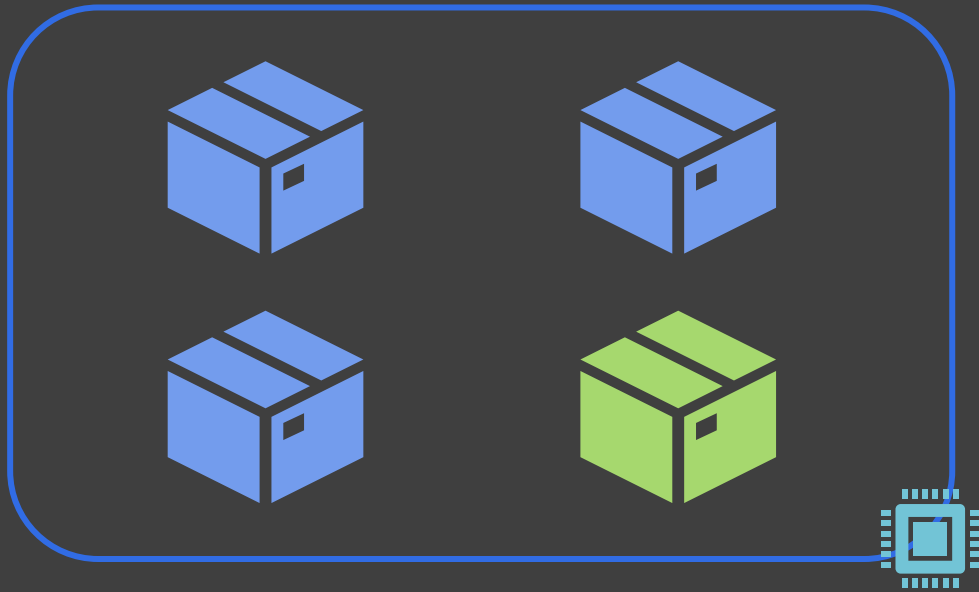
Job A



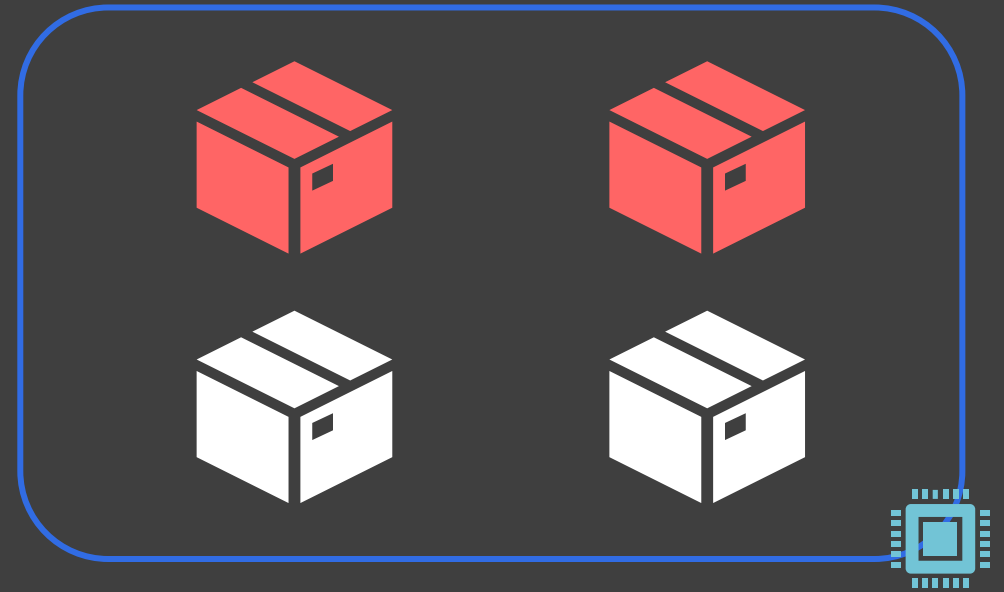
Job B



Job C



GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Scheduling / Migration

Issue

- 虛擬記憶體定址 (Memory Context)：CUDA 程序在啟動時會與特定的硬體 Device ID 綁定。內部的指標和記憶體分頁表是針對 GPU1 的 VRAM 建立的。要搬移，必須要把整塊 VRAM 資料拷貝到 GPU0，並且重新映射所有的記憶體位址，目前 NVIDIA 的驅動程序並不支援這種動態重映射。
- 指令管線狀態：GPU 內部有成千上萬個執行緒的暫存器狀態、運算單元 (SM) 的任務排隊狀態。目前的 MPS Server 並沒有提供「序列化」這些運行中狀態的功能。
- MPS Server 的獨立性：每一張 GPU 都跑著一個獨立的 MPS Control Daemon。Job B 是 GPU1 MPS Server 的一個 Client，它無法在不重新啟動的情況下轉投 GPU0 的 MPS Server 門下。

Slurm Scheduling (1/3)

- 核心的設定軸：

設定	作用	我們目前的值
<code>SchedulerType</code>	主調度器演算法	<code>sched/backfill</code> (預設)
<code>SelectType</code>	資源 fit 演算法	<code>select/cons_tres</code> (GPU/MPS 必需)
<code>SelectTypeParameters</code>	fit 演算法的細部策略 (spread / pack / 各種變體)	<code>CR_Core</code>
<code>PriorityType</code>	job 排序方式	<code>priority/basic</code> (FIFO)
<code>PreemptType</code> / <code>PreemptMode</code>	是否允許 preempt	未設定 = 不 preempt

Slurm Scheduling (2/3)

- 跨節點 bin-packing (把多個小 job 塞到同一個 node)

```
SelectTypeParameters=CR_Core,CR_Pack_Nodes
```



`CR_Pack_Nodes` 讓 `cons_tres` 在 fit 一個 job 時，**優先選已經有負載的 node**，目的是讓沒負載的 node 完整空出來給未來的大 job。預設是 `CR_LL_N` (Least Loaded Nodes) — spread 平均分散，反過來。

對我們這套 (2 GPU node) 的影響很小，因為節點數本來就少。但概念是對的：「優先填已用的卡 → 騰出空卡給大 job」。

- 節點內 GPU bin-packing (多個 mps job 塞同一張卡)：

`select/cons_tres` 的 GRES allocator 預設就會這樣做 — 它會先嘗試把新 job 的 mps slot 塞到「該 node 上 mps 已被部分使用的 GPU」，只有放不下時才動下一張卡。所以對我們的情境，**第一次提交 C 時** Slurm 會自己嘗試 bin-pack；它放不下純粹是因為 GPU0 真的剩下 1 slot < C 的 2 slot 需求。

要進一步調這個層次的細節，可在 `gres.conf` 寫 GPU topology：

```
NodeName=worker-gpu Name=gpu Type=rtx4070 File=/dev/nvidia0 Cores=0-1  
NodeName=worker-gpu Name=gpu Type=rtx4070 File=/dev/nvidia1 Cores=2-3
```



Slurm Scheduling (3/3)

- Custom Slurm Plugin

Slurm Scheduling Strategy (1/2)

- Bin-packing：先塞滿第一張卡，再用第二張卡
- 預設情況下，Slurm 為了散熱和負載平衡，傾向於把 Job 分散到不同的節點。

```
slurm:
  config: |
    SelectType=select/cons_tres
    # 重點：CR_Core_Memory 配合 CR_Pack_Nodes
    # 這會讓 Slurm 優先把任務填入同一個資源塊
    SelectTypeParameters=CR_Core_Memory,CR_Pack_Nodes
```

當 Job B 進來要求 25% MPS 時，Slurm 會檢查 GPU0。如果 GPU0 已經有 Job A (75%)，加起來剛好 100%，它會優先把 Job B 塞進 GPU0。GPU1 就會保持 100% 空閒，留給未來的 Job C。

Slurm Scheduling Strategy (2/2)

- Preemption：需要定義不同的 Partition 優先級
 - HighPrio Partition: 給 Job C (重要大任務)，優先級 10。
 - LowPrio Partition: 給 Job B (小任務)，優先級 1。
- 當 Job C 提交到 HighPrio 時，Slurm 會自動把 GPU1 上的 Job B 踢掉，讓 Job C 執行
- 等到 GPU0 或 GPU1 有空位時，Job B 會自動重新啟動

```
slurm:
  config: |
    PreemptMode=REQUEUE           # 或者用 SUSPEND (如果 Job 支援)
    PreemptType=preempt/partition_prio
    JobRequeue=1
```

Scheduling Strategy

- ~~啟發式排程：改 Slurm 設定即可，快速有效~~
- 數學式排程：尋找最佳解
 - Analytical modeling + optimization
 - 深度強化學習 (DRL)

Analytical modeling + Optimization

- Score Function：給每個 (job, placement) 算分，挑最高的
- 係數 $\alpha, \beta, \gamma, \delta, \varepsilon$ 怎麼定？
 - 線上學習：把 $\alpha - \varepsilon$ 當可學習的權重，用 EXP3 / LinUCB 等簡單演算法線上更新。
 - 深度強化學習 DRL

Score Function

$$\begin{aligned} \text{score}(J, P) = & \alpha \times \text{mps_fit}(J, P) \rightarrow \text{MPS slot 容量配適} \\ & + \beta \times \text{vram_fit}(J, P) \rightarrow \text{VRAM 餘裕} \\ & + \gamma \times \text{topology}(J, P) \rightarrow \text{NCCL 拓撲親和性} \\ & - \delta \times \text{fragmentation}(J, P) \rightarrow \text{放完後碎片化代價} \\ & + \epsilon \times \text{pred_runtime}(J) \rightarrow \text{預測 runtime} \end{aligned}$$

$$\alpha = 0.40, \beta = 0.20, \gamma = 0.0, \delta = 0.20, \epsilon = 0.0$$

Phase6 Roadmap

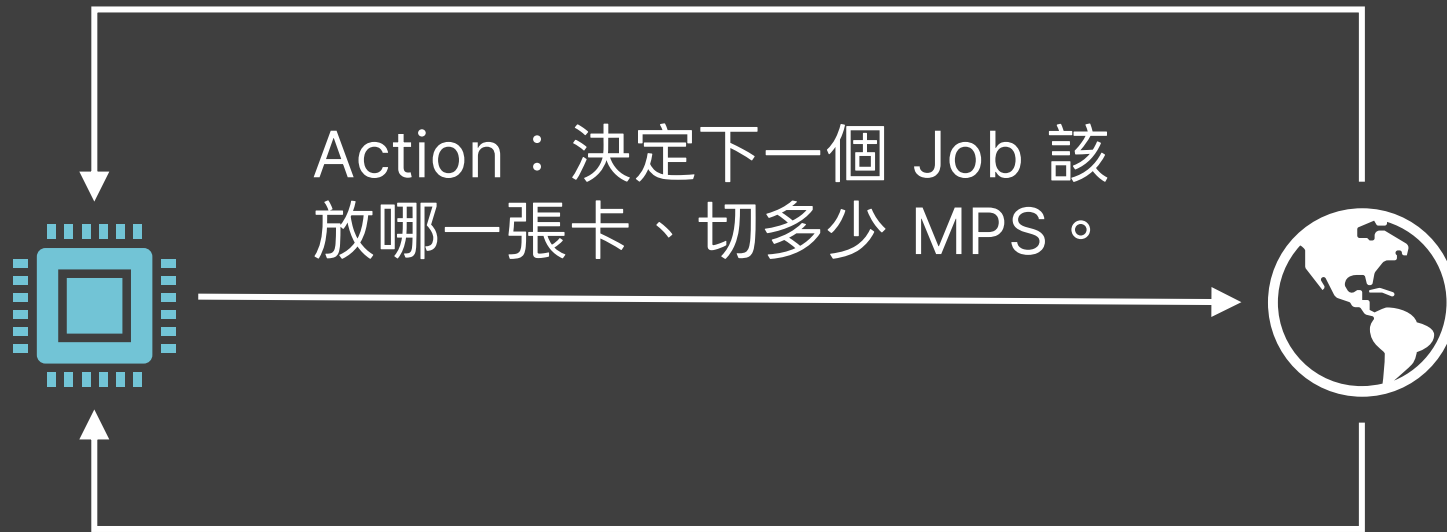
1. 在現有 Slurm + GPU Operator MPS 上，定義打分函數
2. 用 Slurm plugin 把 score 接進 Slurm
3. 在實機跑 10~50 個 job mix 比較 預設設置
4. 把打分函數的 $\alpha - \varepsilon$ 當 RL action，state = cluster snapshot
5. PPO + 公開 trace 訓練原則
6. 對比公式 + RL 與純公式

Contextual Bandit

- 2025 後 ML 領域解決動態決策和個性化推薦的關鍵技術
- 引入了 context 讓算法能夠根據用戶的即時行為或環境狀態做出更精準的動作選擇。
- 這種方法特別適合需要在線學習的場景，例如廣告投放、推薦系統或醫療診斷是核心挑戰。

Deep Reinforcement Learning (DRL)

Reward：如果 Job 跑得快且資源沒浪費，給正分；如果發生 OOM 或排隊太久，給負分。



State：目前叢集所有 GPU 的剩餘資源、等待隊列中的 Job 屬性。

Observability

Observability



雲端服務崛起



錯誤排查難度上升

Observability Aspects

Metrics



負責監控系統有什麼狀況，當要發生服務故障前可以透過設定閾值搭配告警提早得知。

Logs

當問題發生時，可以用來查看故障時正在執行哪些服務，以及產生的錯誤資訊。

Traces

在連續的時間維度上，透過 Trace 以及 Span 關聯，把空間給排列展示出來，並且有 Trace-Context 規範，能夠直觀的看到請求在分散式系統中經過所有服務的紀錄。



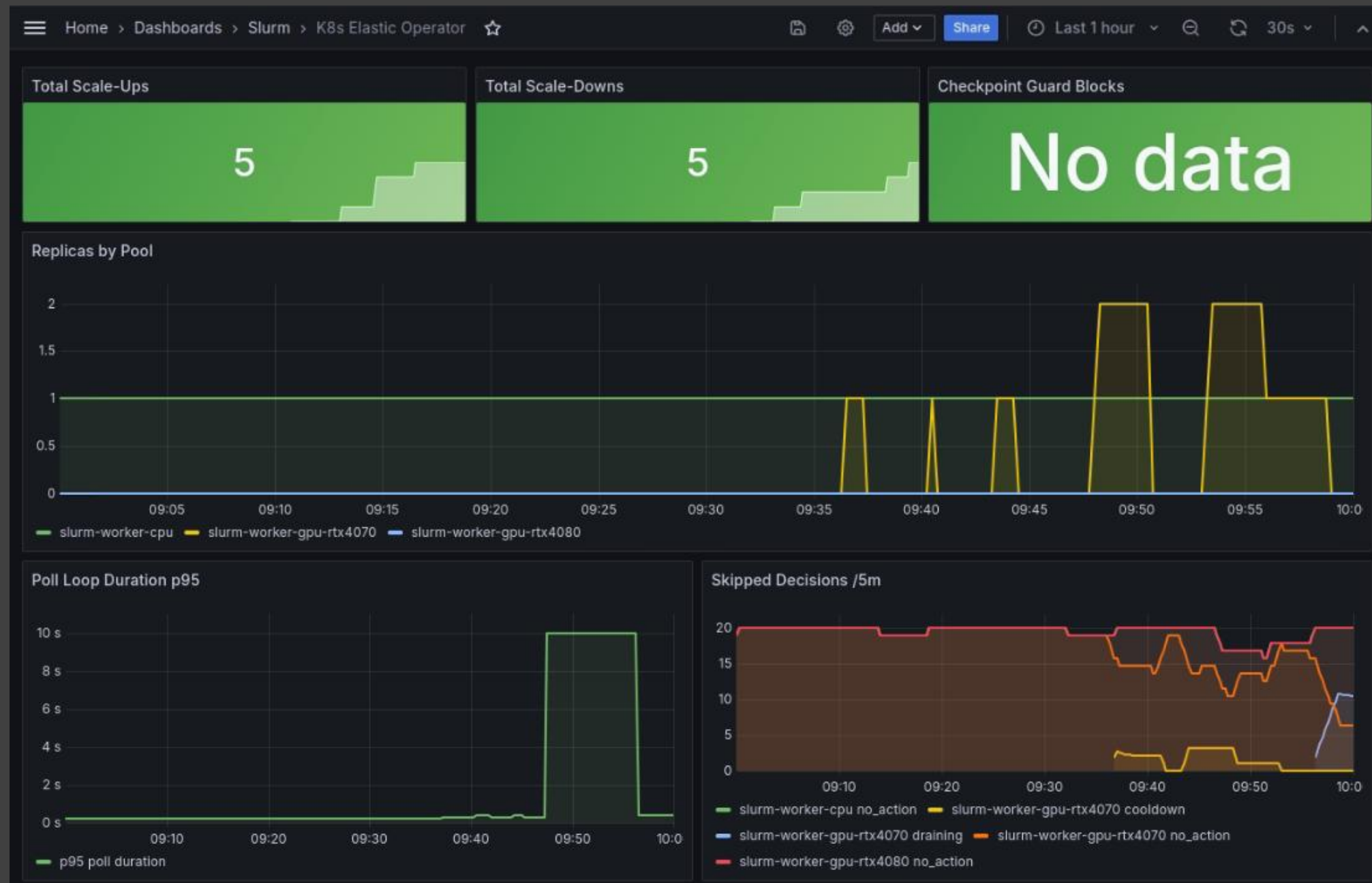
- 最受歡迎的開源監控與告警工具組，是 CNCF 的畢業專案
- 專為動態雲端環境設計，透過 HTTP 拉取 metrics，儲存於時間序列資料庫中，並使用 PromQL 進行數據查詢分析。
- 負責發 Alert 到 Slack/Telegram

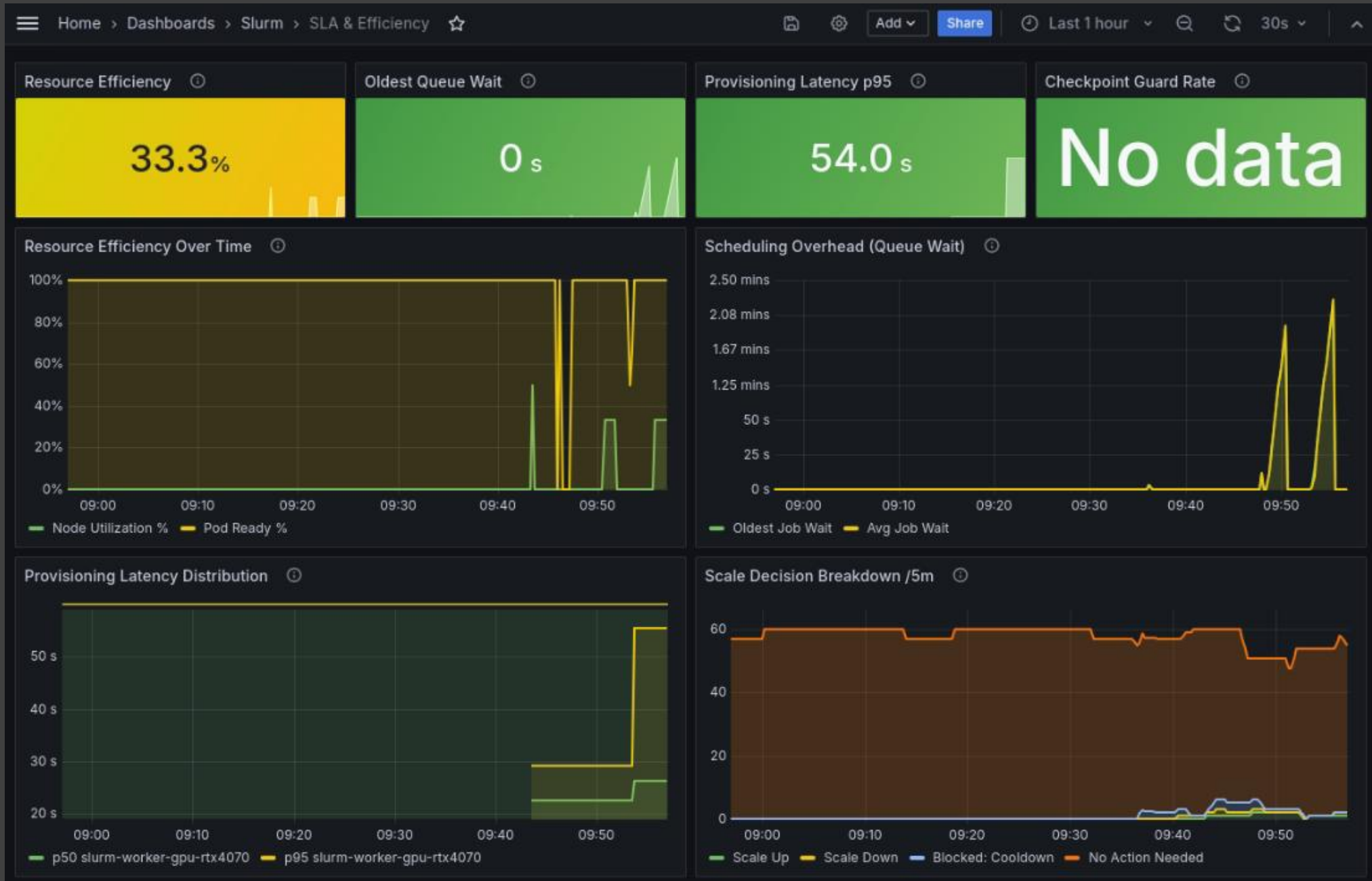


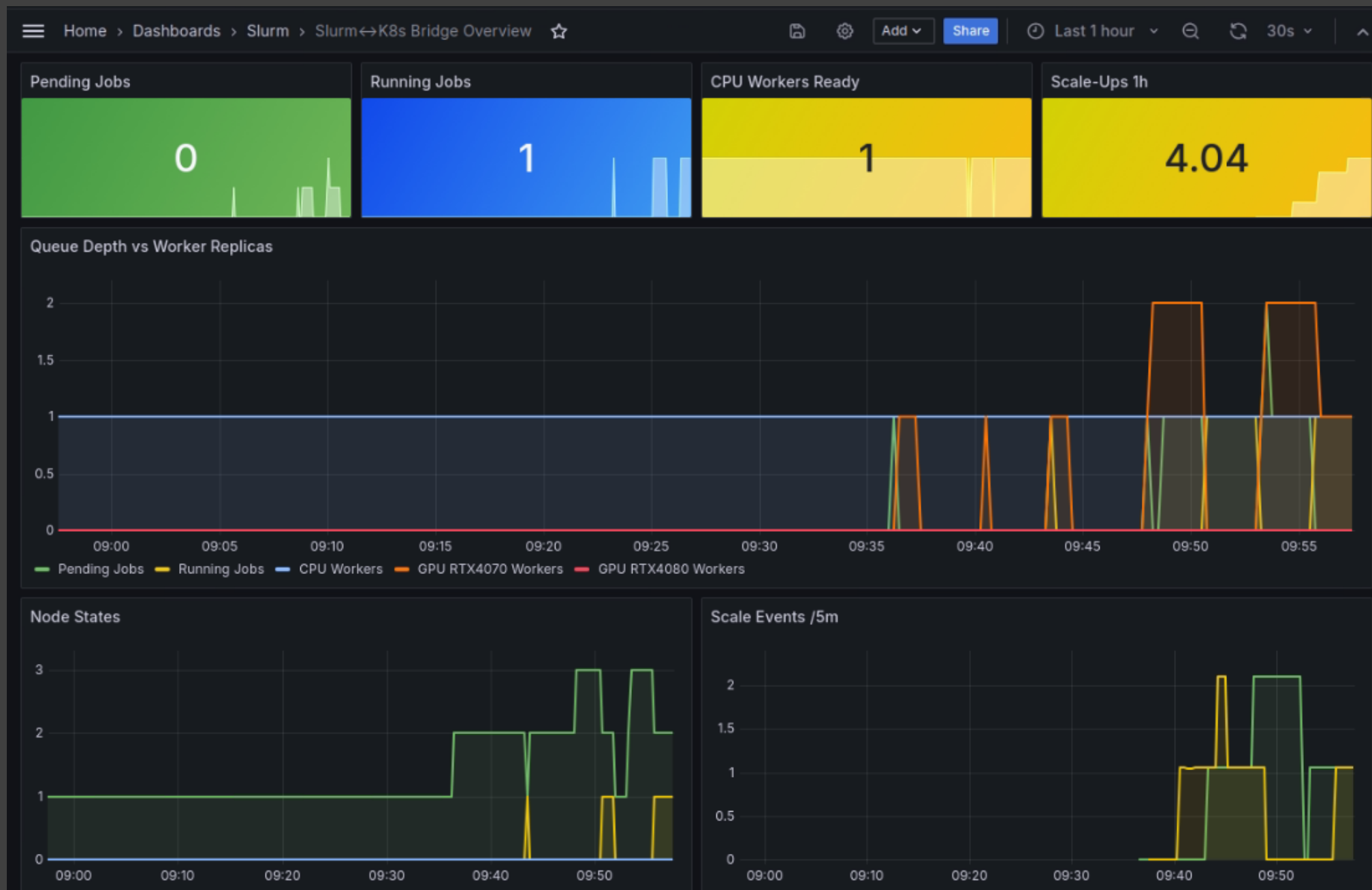
- 能夠將各種資料做成監控儀表板，支援多種資料來源，以及提供豐富的面板選擇，協助我們將資料視覺化。
- Prometheus 預設的 GUI 功能不足，需要 Grafana 這種支援豐富圖表的工具幫助我們可視化 Metrics 資訊。



- 由 CNCF 孵化的開源專案，是 Cloud Native 的可觀測性框架
- 提供標準化的 API、SDK 與協議自動檢測、蒐集、導出遙測數據資料 (Metrics、Log、Trace)，並支援 W3C 定義的 Http trace-context 規範

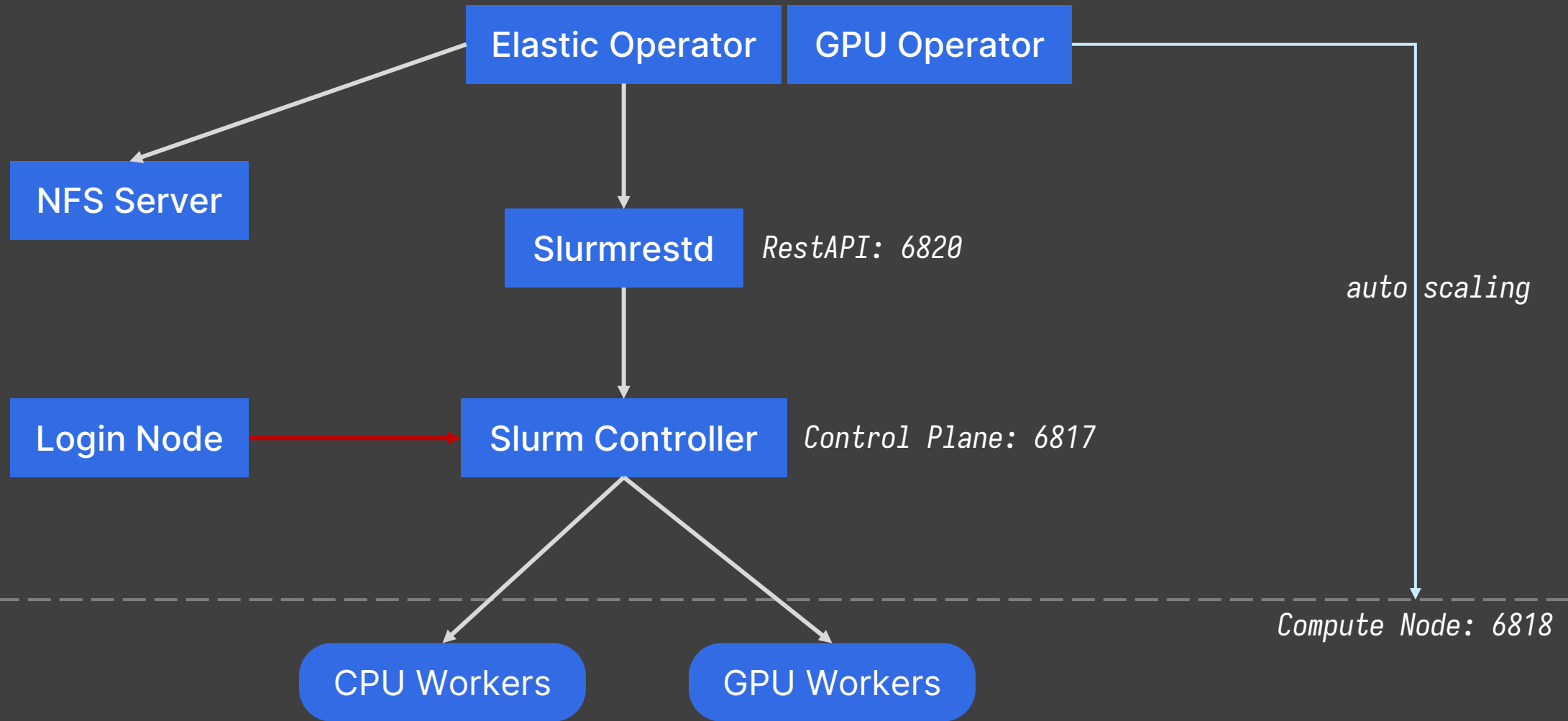






Networking

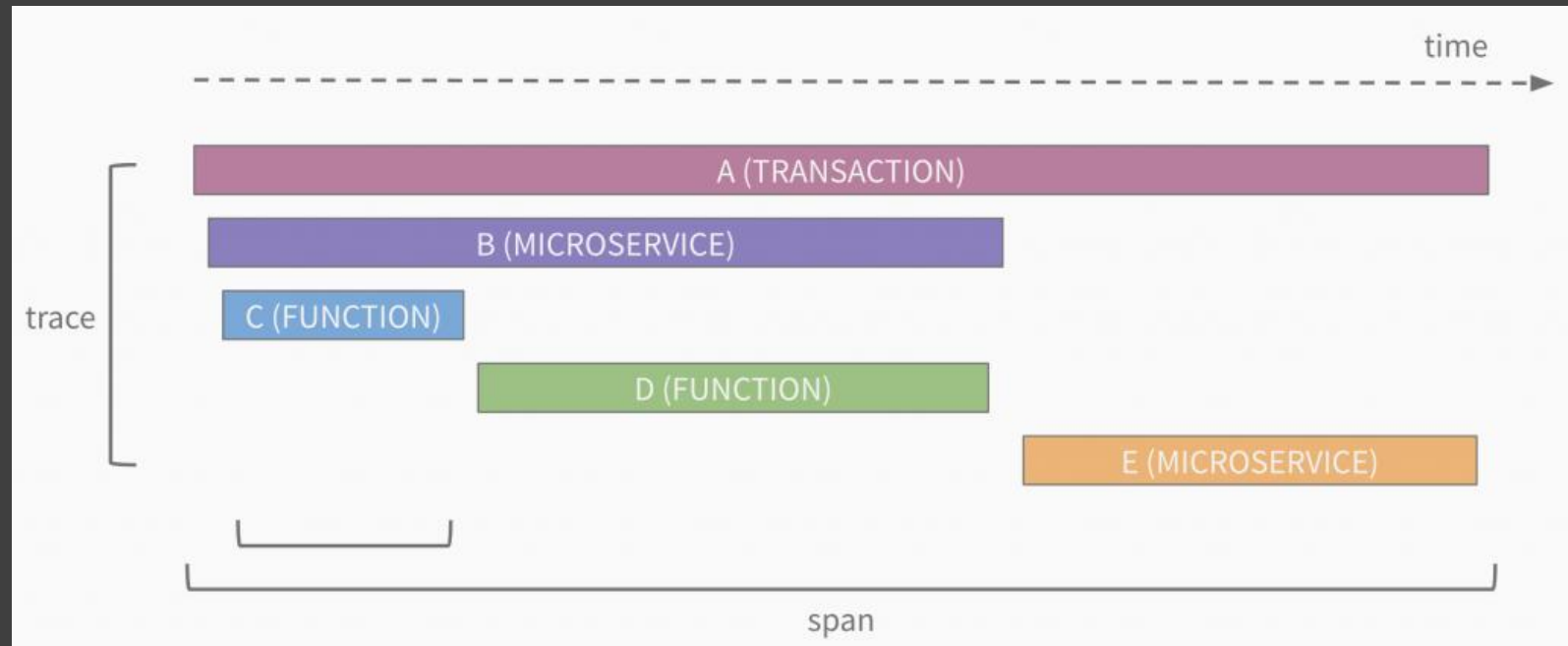
Network Flow



Future Work

Distributed Tracing (1/2)

- 搜集應用程式內部與端到端請求相關的資料，以實現對整個系統的監控和分析。
 - 服務名稱
 - 操作名稱
 - 持續時間



Distributed Tracing (2/2)

- 讓一個 Job 的完整生命週期變成一條可視化的 Trace :

```
[sbatch submit] → [pending in queue] → [scale-up decision]  
→ [K8s provisioning] → [slurmd registration] → [DDP torchrun]  
→ [checkpoint write] → [scale-down decision] → [job complete]
```

- 紀錄 job_id, pool, checkpoint_age, time 等屬性

(+) Tools : OpenTelemetry + Grafana Tempo

TODO

- Job order (process sched.) *** -> job migration (formula?)
- 多個 job 怎麼分配到硬體 (OK)
 - 使用者描述需求
 - 不同硬體的 SM 要切
- 網路頻寬 **
 - 延遲 metric (job submitted -> completed)
 - 考量硬體規格
- 根據數學公式 * (working)
 - 深度強化學習分配？